

Statistical Computing

Bootstrap and Jackknife

Spring 2026

The Bootstrap

The Jackknife

Bootstrap Confidence Intervals

Application: Cross Validation

The Bootstrap

Bootstrap methods

Bootstrap methods are a class of *nonparametric Monte Carlo* methods that estimate the distribution of a population by *resampling*.

- Resampling methods treat an observed sample as a *finite population*, and random samples are generated (resampled) from it to estimate population characteristics and make inferences.
- Bootstrap methods are often used when the distribution of the target population is not specified; the sample is the only information available.
- The term “bootstrap” here refers to *nonparametric* bootstrap (sampling from the data); Monte Carlo methods that sample from a fully specified distribution are called *parametric* bootstrap.

- Let $\mathbf{x} = (x_1, \dots, x_n)$ be an observed random sample from a distribution with cdf $F(x)$.

Bootstrap resampling

If X^* is selected at random from $\mathbf{x}$, then

$$\mathbb{P}(X^* = x_i) = \frac{1}{n}, \quad i = 1, \dots, n,$$

and a bootstrap sample $X_1^*, \dots, X_n^*$ is generated by sampling *with replacement* from $\mathbf{x}$. The variables X_i^* are iid, uniformly distributed on $\{x_1, \dots, x_n\}$.

Empirical distribution function (recall)

Definition

An estimator of $F(x) = \mathbb{P}(X \leq x)$ is the proportion of sample points that fall in $(-\infty, x]$. The empirical cdf (ecdf) is

$$F_n(x) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{X_i \leq x\}.$$

Piecewise form

Let $x_{(1)} \leq \cdots \leq x_{(n)}$ be the ordered sample. Then

$$F_n(x) = \begin{cases} 0, & x < x_{(1)}, \\ \frac{i}{n}, & x_{(i)} \leq x < x_{(i+1)}, \quad i = 1, \dots, n-1, \\ 1, & x_{(n)} \leq x. \end{cases}$$

Two approximations in bootstrap

$F_n(x)$ is itself the distribution function of a random variable uniformly distributed on $\{x_1, \dots, x_n\}$. In bootstrap, there are two approximations:

- $F_n(x)$ approximates $F(x)$.
- The ecdf of the bootstrap replicates, $\hat{F}^*$, approximates $F_n(x)$.

$$F \longrightarrow X \longrightarrow F_n, \quad F_n \longrightarrow X^* \longrightarrow \hat{F}^*.$$

- Resampling from $\mathbf{x}$ is equivalent to generating random samples from $F_n(x)$.

Bootstrap sampling procedure

Let θ be the parameter of interest and $\hat{\theta}$ an estimator.

1. For each replicate $b = 1, \dots, B$:
 - 1.1 Generate $\mathbf{x}^{*(b)}$ by sampling with replacement from $\mathbf{x}$.
 - 1.2 Compute $\hat{\theta}^{(b)}$ from $\mathbf{x}^{*(b)}$.
2. The bootstrap estimate of the distribution of $\hat{\theta}$ is the empirical distribution of $\hat{\theta}^{(1)}, \dots, \hat{\theta}^{(B)}$.

Example: F_n and bootstrap samples

- Observed sample:

$$\mathbf{x} = \{2, 2, 1, 1, 5, 4, 4, 3, 1, 2\}.$$

- Resampling from $\mathbf{x}$ selects 1, 2, 3, 4, 5 with probabilities 0.3, 0.3, 0.1, 0.2, 0.1, respectively.
- Therefore the cdf of a randomly selected replicate equals the ecdf $F_n(x)$:

$$F_n(x) = \begin{cases} 0, & x < 1, \\ 0.3, & 1 \leq x < 2, \\ 0.6, & 2 \leq x < 3, \\ 0.7, & 3 \leq x < 4, \\ 0.9, & 4 \leq x < 5, \\ 1, & x \geq 5. \end{cases}$$

- Note: if F_n is not close to F , the distribution of replicates will not be close to F .

Bootstrap estimate of standard error

Bootstrap standard error

The bootstrap estimate of standard error of an estimator $\hat{\theta}$ is the sample standard deviation of the bootstrap replicates $\hat{\theta}^{(1)}, \dots, \hat{\theta}^{(B)}$:

$$\widehat{\text{se}}_B(\hat{\theta}^*) = \sqrt{\frac{1}{B-1} \sum_{b=1}^B (\hat{\theta}^{(b)} - \bar{\theta}^*)^2}, \quad \bar{\theta}^* = \frac{1}{B} \sum_{b=1}^B \hat{\theta}^{(b)}.$$

- The number of replicates needed for good estimates of standard error is typically not large: $B = 50$ is usually large enough, and rarely is $B > 200$ necessary.
- (Much larger B will be needed for confidence interval estimation.)

Example: law school data (correlation)

- Data: LSAT and GPA for $n = 15$ law schools (random sample from a universe of 82 law schools).
- Goal: estimate the correlation between LSAT and GPA, and compute the bootstrap estimate of the standard error of the sample correlation.

LSAT: 576, 635, 558, 578, 666, 580, 555, 661, 651, 605, 653, 575, 545, 572, 594

GPA: 339, 330, 281, 303, 344, 307, 300, 343, 336, 313, 312, 274, 276, 288, 296

Bootstrap standard error for correlation (Python)

Code

```
import numpy as np
import matplotlib.pyplot as plt

# data
lsat = np.array([576, 635, 558, 578, 666, 580, 555, 661, 651, 605,
                 653, 575, 545, 572, 594], dtype=float)
gpa = np.array([339, 330, 281, 303, 344, 307, 300, 343, 336, 313,
                312, 274, 276, 288, 296], dtype=float)

def corr(x, y):
    return np.corrcoef(x, y)[0, 1]

# observed statistic
R0 = corr(lsat, gpa)
```

Bootstrap standard error for correlation (Python)

Code

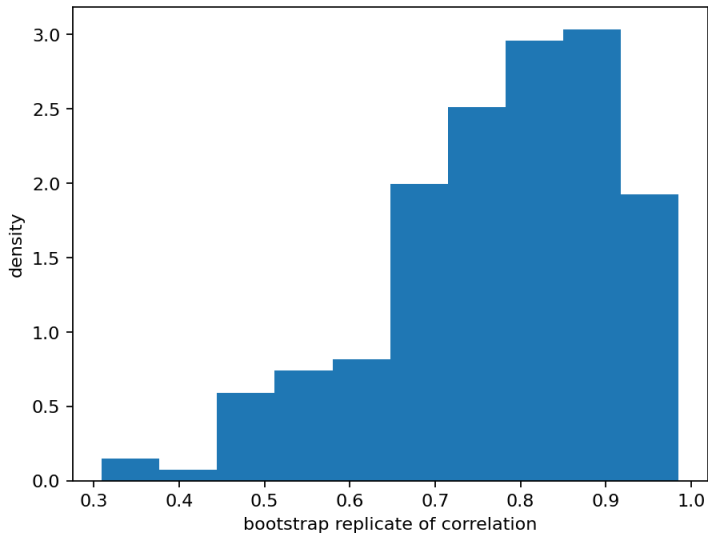
```
# nonparametric bootstrap
B = 200; n = len(lsat)
rng = np.random.default_rng(2026)

R = np.empty(B)
for b in range(B):
    i = rng.integers(0, n, size=n) # sample indices with replacement
    R[b] = corr(lsat[i], gpa[i])

print("R0 = %.4f, se_boot = %.4f"%(R0, R.std(ddof=1)))

# histogram of bootstrap replicates
plt.hist(R, density=True, bins=10)
plt.xlabel("bootstrap replicate of correlation")
plt.ylabel("density")
plt.show()
```

Figure: histogram of bootstrap replicates



Bias

If $\hat{\theta}$ is an unbiased estimator of θ , then $\mathbb{E}[\hat{\theta}] = \theta$. The bias of an estimator $\hat{\theta}$ for θ is

$$\text{bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta} - \theta] = \mathbb{E}[\hat{\theta}] - \theta.$$

- Every statistic is an unbiased estimator of its expected value. In particular, the sample mean is an unbiased estimator of the population mean.
- Example of a biased estimator: the MLE of variance,

$$\hat{\sigma}^2 = \frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})^2, \quad \mathbb{E}[\hat{\sigma}^2] = \left(1 - \frac{1}{n}\right) \sigma^2,$$

so $\hat{\sigma}^2$ underestimates σ^2 and the bias is $-\sigma^2/n$.

Bootstrap estimate of bias

- Bootstrap estimation of bias uses bootstrap replicates of $\hat{\theta}$ to estimate the sampling distribution of $\hat{\theta}$.
- For the finite population $\mathbf{x} = (x_1, \dots, x_n)$, the parameter is $\hat{\theta}(\mathbf{x})$ and we have B iid bootstrap replicates $\hat{\theta}^{(b)}$.

Bootstrap bias estimate

Let

$$\bar{\theta}^* = \frac{1}{B} \sum_{b=1}^B \hat{\theta}^{(b)}, \quad \hat{\theta} = \hat{\theta}(\mathbf{x}).$$

The bootstrap estimate of bias is

$$\widehat{\text{bias}}(\hat{\theta}) = \bar{\theta}^* - \hat{\theta}.$$

(Here, θ is replaced by $\hat{\theta}$ because bootstrap samples F_n in place of F .)

Example: bias of the sample correlation (law school data)

- Using the law school data from the previous example, compute the bootstrap estimate of bias in the sample correlation.
- Use $B = 2000$ bootstrap replicates (larger B is used for estimating bias).

Rule of thumb: if $|\text{bias}|/\text{se} \leq 0.25$, it is not usually necessary to adjust for bias. Here the bias is small relative to the standard error ($\text{cv} < 0.1$), so adjustment is not necessary.

Bootstrap bias for correlation (Python)

Code

```
# sample estimate
theta_hat = corr(lsat, gpa)

# bootstrap estimate of bias
B = 2000
n = len(lsat)

theta_b = np.empty(B)
for b in range(B):
    i = rng.integers(0, n, size=n) # resample indices
    theta_b[b] = corr(lsat[i], gpa[i])

bias_hat = theta_b.mean() - theta_hat
se_hat = theta_b.std(ddof=1)
cv = bias_hat / se_hat
print(bias_hat, se_hat, cv)
```

The Jackknife

The jackknife: overview

- The jackknife is another resampling method:
- The jackknife is like a “leave-one-out” type of cross-validation.

Jackknife samples and replicates

Let $\mathbf{x} = (x_1, \dots, x_n)$ be an observed random sample. Define the i th jackknife sample by leaving out x_i :

$$\mathbf{x}^{(i)} = (x_1, \dots, x_{i-1}, x_{i+1}, \dots, x_n).$$

If $\hat{\theta} = T_n(\mathbf{x})$, define the i th jackknife replicate

$$\hat{\theta}^{(i)} = T_{n-1}(\mathbf{x}^{(i)}), \quad i = 1, \dots, n.$$

Bias estimate

Let

$$\hat{\theta}^{(\cdot)} = \frac{1}{n} \sum_{i=1}^n \hat{\theta}^{(i)}, \quad \hat{\theta} = \hat{\theta}(\mathbf{x}).$$

The jackknife estimate of bias is

$$\widehat{\text{bias}}_{\text{jack}} = (n - 1) \left(\hat{\theta}^{(\cdot)} - \hat{\theta} \right).$$

- The jackknife requires only n replications to estimate bias.
- The bootstrap estimate of bias typically requires several hundred replicates.

Why the factor $(n - 1)$ appears

- Consider the case where θ is the population variance σ_X^2 . The plug-in estimate is

$$\hat{\theta} = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2, \quad \text{bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta} - \sigma_X^2] = -\frac{\sigma_X^2}{n}.$$

- Each jackknife replicate uses sample size $n - 1$, so its bias is $-\sigma_X^2 / (n - 1)$.
- Therefore

$$\mathbb{E}[\hat{\theta}^{(i)} - \hat{\theta}] = \text{bias}(\hat{\theta}^{(i)}) - \text{bias}(\hat{\theta}) = -\frac{\sigma_X^2}{n(n-1)} = \frac{\text{bias}(\hat{\theta})}{n-1},$$

so multiplying the leave-one-out difference by $(n - 1)$ yields the correct bias for this plug-in estimator.

Code

```
import numpy as np

x = np.arange(1, 6) # 1..5
for i in range(len(x)):
    print(np.delete(x, i)) # leave out the i-th element
```

Example: jackknife bias (patch data)

- Data: hormone measurements for $n = 8$ subjects after wearing a medical patch.
- Compute the jackknife estimate of bias for the patch data (ratio statistic)

$$\hat{\theta} = \frac{\bar{y}}{\bar{z}}.$$

subject	placebo	oldpatch	newpatch	z	y
1	9243	17649	16449	8406	-1200
2	9671	12013	14614	2342	2601
3	11792	19979	17274	8187	-2705
4	13357	21816	23798	8459	1982
5	9055	13850	12560	4795	-1290
6	6290	9806	10157	3516	351
7	12412	17208	16570	4796	-638
8	18806	29044	26325	10238	-2719

Jackknife bias for ratio statistic (Python)

Code

```
# patch data columns
y = np.array([-1200, 2601, -2705, 1982, -1290, 351, -638, -2719], dtype=float)
z = np.array([ 8406, 2342, 8187, 8459, 4795, 3516, 4796, 10238], dtype=float)

n = len(y)
theta_hat = y.mean() / z.mean()

# jackknife replicates
theta_jack = np.empty(n)
for i in range(n):
    theta_jack[i] = np.delete(y, i).mean() / np.delete(z, i).mean()

bias_jack = (n - 1) * (theta_jack.mean() - theta_hat)
print(bias_jack)
```

Standard error estimate

A jackknife estimate of standard error is

$$\hat{\text{se}}_{\text{jack}} = \sqrt{\frac{n-1}{n} \sum_{i=1}^n (\hat{\theta}^{(i)} - \hat{\theta}^{(\cdot)})^2}.$$

- The factor $(n-1)/n$ makes $\hat{\text{se}}_{\text{jack}}$ an unbiased estimator of the standard error of the sample mean.

Jackknife se for ratio statistic (Python)

Code

```
y = np.array([-1200, 2601, -2705, 1982, -1290, 351, -638, -2719], dtype=float)
z = np.array([ 8406, 2342, 8187, 8459, 4795, 3516, 4796, 10238], dtype=float)

n = len(y)
theta_hat = y.mean() / z.mean()

theta_jack = np.empty(n)
for i in range(n):
    theta_jack[i] = np.delete(y, i).mean() / np.delete(z, i).mean()

bias_jack = (n - 1) * (theta_jack.mean() - theta_hat)
se_jack = np.sqrt((n - 1) * np.mean((theta_jack - theta_jack.mean())**2))
cv = bias_jack / se_jack

print(se_jack, cv)
```

When the jackknife fails

- The jackknife can fail when the statistic $\hat{\theta}$ is not “smooth.”
- Smoothness means that small changes in the data correspond to small changes in the statistic.
- The median is an example of a statistic that is not smooth.

Failure example: median (Python)

Code

```
x = np.array([29, 79, 41, 86, 91, 5, 50, 83, 51, 42], dtype=float); n = len(x)

# jackknife estimate of se of the median
M = np.empty(n)
for i in range(n):
    M[i] = np.median(np.delete(x, i))
Mbar = M.mean()
se_jack = np.sqrt((n - 1) / n * np.sum((M - Mbar)**2))
print(se_jack)

# bootstrap estimate of se of the median
B = 1000
Mb = np.empty(B)
for b in range(B):
    y = rng.choice(x, size=n, replace=True)
    Mb[b] = np.median(y)
print(Mb.std(ddof=1))
```

Details and results (as shown in the text)

- The sample:

x : 29, 79, 41, 86, 91, 5, 50, 83, 51, 42

- Jackknife medians:

51, 50, 51, 50, 50, 51, 51, 50, 50, 51

- Jackknife estimate of se: 1.5.
- Bootstrap estimate of se (from replicates of the median): 13.69387.
- The jackknife fails here because the median is not smooth.

Bootstrap Confidence Intervals

- In this section, several approaches to obtaining approximate confidence intervals for the target parameter in a bootstrap are discussed.
- Methods include:
 - the standard normal bootstrap confidence interval,
 - the basic bootstrap confidence interval,
 - the bootstrap percentile confidence interval.

Standard normal bootstrap confidence interval

If $\hat{\theta}$ is approximately normal and unbiased for θ , then

$$Z = \frac{\hat{\theta} - \theta}{\text{se}(\hat{\theta})} \sim N(0, 1),$$

giving the approximate $100(1 - \alpha)\%$ confidence interval

Z-interval

$$\hat{\theta} \pm z_{\alpha/2} \text{se}(\hat{\theta}), \quad z_{\alpha/2} = \Phi^{-1}(1 - \alpha/2).$$

In practice $\text{se}(\hat{\theta})$ is unknown. The *bootstrap normal CI* plugs in the bootstrap SE estimate:

$$\hat{\theta} \pm z_{\alpha/2} \widehat{\text{se}}_B(\hat{\theta}^*),$$

where $\widehat{\text{se}}_B$ is the sample standard deviation of the bootstrap replicates $\hat{\theta}^{(1)}, \dots, \hat{\theta}^{(B)}$.

Assumptions and remarks (normal CI)

- This interval is easy to compute, but it relies on assumptions:
 - the distribution of $\hat{\theta}$ is approximately normal;
 - $\hat{\theta}$ is unbiased for θ .
- These assumptions do not always hold — especially when n is small or $\hat{\theta}$ is not a simple mean.
- The basic and percentile intervals (next) do not require normality of $\hat{\theta}$.

Basic bootstrap confidence interval

- The basic bootstrap confidence interval (pivotal CI) transforms the distribution of the replicates by subtracting the observed statistic.
- The quantiles of the transformed sample $\hat{\theta}^* - \hat{\theta}$ are used to determine confidence limits.

Basic (pivotal) CI

Let $\hat{\theta}_q^*$ denote the q sample quantile of the bootstrap replicates $\hat{\theta}^*$. Then the $100(1 - \alpha)\%$ basic bootstrap confidence limits are

$$\left(2\hat{\theta} - \hat{\theta}_{1-\alpha/2}^*, 2\hat{\theta} - \hat{\theta}_{\alpha/2}^* \right).$$

Derivation idea (basic CI)

- Observe a simple random sample $x_1, \dots, x_n$, and compute $\hat{\theta} = \hat{\theta}(x_1, \dots, x_n)$.
- Generate B replicates $\hat{\theta}^*$ from an ordinary bootstrap.
- For a symmetric $100(1 - \alpha)\%$ confidence interval (L, U) for θ based on bootstrap replicates, require

$$\Pr(L > \theta) = \Pr(U < \theta) = \alpha/2.$$

Derivation steps (basic CI)

- For the lower limit:

$$\alpha/2 = \Pr(L > \theta) = \Pr(L - \hat{\theta} > \theta - \hat{\theta}) = \Pr(\hat{\theta} - \theta > \hat{\theta} - L).$$

- Hence $1 - \alpha/2 = \Pr(\hat{\theta} - \theta \leq \hat{\theta} - L)$, so $\hat{\theta} - L$ is the $(1 - \alpha/2)$ quantile of $\hat{\theta} - \theta$.
- These exact percentiles are unknown, but can be estimated from the bootstrap replicates:

$$\hat{\theta}_{1-\alpha/2}^* - \hat{\theta} \approx \text{the } (1 - \alpha/2) \text{ quantile of } (\hat{\theta} - \theta).$$

- Set $\hat{\theta} - L = \hat{\theta}_{1-\alpha/2}^* - \hat{\theta}$.

Conclusion (basic CI)

- Similarly, solving $\alpha/2 = \Pr(U < \theta)$ leads to $\hat{\theta} - U = \hat{\theta}_{\alpha/2}^* - \hat{\theta}$.

Basic CI (final form)

$$(L, U) = \left(2\hat{\theta} - \hat{\theta}_{1-\alpha/2}^*, 2\hat{\theta} - \hat{\theta}_{\alpha/2}^* \right).$$

Percentile bootstrap confidence interval

- A bootstrap percentile interval uses the empirical distribution of the bootstrap replicates as the reference distribution.
- The quantiles of this empirical distribution estimate the quantiles of the sampling distribution of $\hat{\theta}$.
- This can match the true distribution better when the distribution of $\hat{\theta}$ is not normal.

Percentile CI

Let $\hat{\theta}^{*(1)}, \dots, \hat{\theta}^{*(B)}$ be bootstrap replicates of $\hat{\theta}$. Compute the $\alpha/2$ quantile $\hat{\theta}_{\alpha/2}^*$ and the $(1 - \alpha/2)$ quantile $\hat{\theta}_{1-\alpha/2}^*$ from the ecdf of the replicates. The percentile interval is

$$(\hat{\theta}_{\alpha/2}^*, \hat{\theta}_{1-\alpha/2}^*).$$

Example: confidence intervals for patch ratio statistic

- Compute 95% confidence intervals for the ratio statistic in the patch data:

$$\hat{\theta} = \frac{\bar{y}}{\bar{z}}.$$

- Use $B = 2000$ ordinary bootstrap replicates of $\hat{\theta}$.
- Compute the normal, basic, and percentile intervals from their definitions (using the bootstrap replicates).

Patch ratio: bootstrap CIs from definitions (Python)

Code

```
from scipy.stats import norm

# bootstrap replicates
t = np.empty(B)
for b in range(B):
    i = rng.integers(0, n, size=n)
    t[b] = y[i].mean() / z[i].mean()

# bootstrap summaries
bias_hat = t.mean() - theta_hat; se_hat = t.std(ddof=1)
```

Patch ratio: bootstrap CIs from definitions (Python)

Code

```
alpha = np.array([0.025, 0.975])

# normal interval (definition-based; bias correction is noted separately in the text)
ci_norm = theta_hat + norm.ppf(alpha) * se_hat
# basic interval
ci_basic = 2 * theta_hat - np.quantile(t, alpha[::-1]) # [0.975, 0.025]
# percentile interval
ci_perc = np.quantile(t, alpha)

print({"t0": theta_hat, "bias": bias_hat, "se": se_hat})
print("normal :", ci_norm)
print("basic :", ci_basic)
print("percentile :", ci_perc)
```

Application: Cross Validation

- Cross validation is a data partitioning method for assessing model fit, prediction accuracy, or classifier performance.
- **K -fold CV**: divide data into K folds; use each fold as the test set in turn while fitting on the remaining $K - 1$ folds. The model is fitted K times.
- **Leave-one-out CV** (LOOCV): special case $K = n$, each observation serves as a test point once.
 - Equivalent to the jackknife.

Model selection example: ironslag data

- Data: ironslag (DAAG), $n = 53$ measurements of iron content by two methods:
 - chemical measurement (X),
 - magnetic measurement (Y).
- A scatterplot suggests a positive association, but the relation may not be linear.

Four candidate models for predicting Y from X :

1. Linear: $Y = \theta_0 + \theta_1 X + \varepsilon$.
2. Quadratic: $Y = \theta_0 + \theta_1 X + \theta_2 X^2 + \varepsilon$.
3. Exponential: $\log(Y) = \log(\theta_0) + \theta_1 X + \varepsilon$.
4. Log-Log: $\log(Y) = \theta_0 + \theta_1 \log(X) + \varepsilon$.

Fit four models and plot fitted curves (Python)

Code

```
import pandas as pd

X = pd.read_csv("data/chemical.csv").values.flatten()
Y = pd.read_csv("data/magnetic.csv").values.flatten()

a = np.arange(10.0, 40.0 + 1e-12, 0.1)
# ---- Model 1: linear  $Y = b_0 + b_1 X$ 
b1 = np.polyfit(X, Y, deg=1) # [b1, b0]
yhat1 = b1[1] + b1[0] * a
# ---- Model 2: quadratic  $Y = b_0 + b_1 X + b_2 X^2$ 
b2 = np.polyfit(X, Y, deg=2) # [b2, b1, b0]
yhat2 = b2[2] + b2[1] * a + b2[0] * a**2
# ---- Model 3: exponential  $\log(Y) = c_0 + c_1 X \Rightarrow Y = \exp(c_0 + c_1 X)$ 
c3 = np.polyfit(X, np.log(Y), deg=1) # [c1, c0]
yhat3 = np.exp(c3[1] + c3[0] * a)
# ---- Model 4: log-log  $\log(Y) = d_0 + d_1 \log(X)$ 
d4 = np.polyfit(np.log(X), np.log(Y), deg=1) # [d1, d0]
```

Fit four models and plot fitted curves (Python)

Code

```
fig, ax = plt.subplots(2, 2)

ax[0,0].scatter(X, Y, s=18)
ax[0,0].plot(a, yhat1, linewidth=2)
ax[0,0].set_title("Linear")
ax[0,0].set_xlabel("chemical"); ax[0,0].set_ylabel("magnetic")

ax[0,1].scatter(X, Y, s=18)
ax[0,1].plot(a, yhat2, linewidth=2)
ax[0,1].set_title("Quadratic")
ax[0,1].set_xlabel("chemical"); ax[0,1].set_ylabel("magnetic")
```

Fit four models and plot fitted curves (Python)

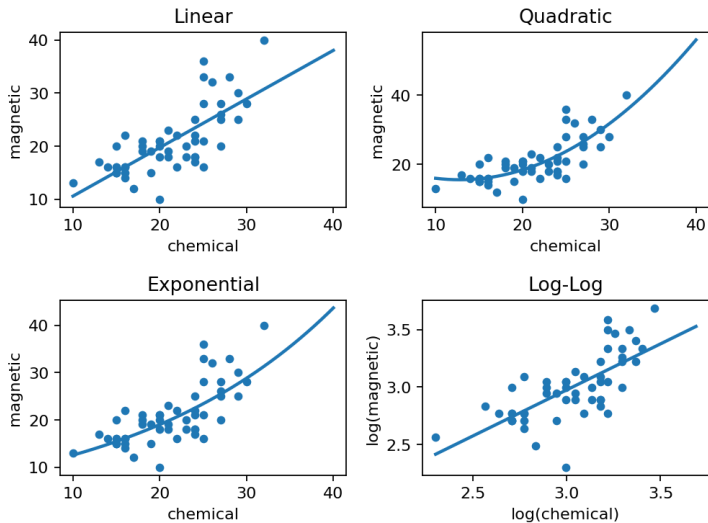
Code

```
ax[1,0].scatter(X, Y, s=18)
ax[1,0].plot(a, yhat3, linewidth=2)
ax[1,0].set_title("Exponential")
ax[1,0].set_xlabel("chemical"); ax[1,0].set_ylabel("magnetic")

ax[1,1].scatter(np.log(X), np.log(Y), s=18)
ax[1,1].plot(np.log(a), d4[1] + d4[0] * np.log(a), linewidth=2)
ax[1,1].set_title("Log-Log")
ax[1,1].set_xlabel("log(chemical)"); ax[1,1].set_ylabel("log(magnetic)")

plt.tight_layout()
plt.show()
```


Figure: four proposed models



Procedure (n -fold leave-one-out cross validation):

1. For $k = 1, \dots, n$, let (x_k, y_k) be the test point and fit the model on the remaining $n - 1$ observations.
 - 1.1 Compute the predicted response $\hat{y}_k$ for the test point.
 - 1.2 Compute the prediction error $e_k = y_k - \hat{y}_k$.
2. Estimate prediction error:

$$\hat{\sigma}^2 = \frac{1}{n} \sum_{k=1}^n e_k^2.$$

Model selection by LOOCV (Python)

Code

```
n = len(Y)
e1 = np.empty(n); e2 = np.empty(n); e3 = np.empty(n); e4 = np.empty(n)
for k in range(n):
    x_tr = np.delete(X, k)
    y_tr = np.delete(Y, k)
    b1 = np.polyfit(x_tr, y_tr, deg=1) # Model 1: linear
    yhat1 = b1[1] + b1[0] * X[k]; e1[k] = Y[k] - yhat1
    b2 = np.polyfit(x_tr, y_tr, deg=2) # Model 2: quadratic
    yhat2 = b2[2] + b2[1] * X[k] + b2[0] * X[k]**2; e2[k] = Y[k] - yhat2
    c3 = np.polyfit(x_tr, np.log(y_tr), deg=1) # Model 3: exponential (fit log(Y) on X)
    yhat3 = np.exp(c3[1] + c3[0] * X[k]); e3[k] = Y[k] - yhat3
    d4 = np.polyfit(np.log(x_tr), np.log(y_tr), deg=1) # Model 4: log-log (fit log(Y) on log(X))
    yhat4 = np.exp(d4[1] + d4[0] * np.log(X[k])); e4[k] = Y[k] - yhat4
mse = np.array([np.mean(e1**2), np.mean(e2**2), np.mean(e3**2), np.mean(e4**2)])
print(mse)
```

- The following estimates for prediction error are obtained from the n -fold (leave-one-out) cross validation:

$(19.55644, 17.85248, 18.44188, 20.45424)$.

- According to the prediction error criterion, Model 2 (quadratic) would be the best fit for the data.